EngineAI T800 Humanoid Robot

Technical Questionnaire Responses & System Integration Guide

Document Ref: ENG-T800-TQ-2026-ANS | Target Platform: EngineAI T800 Humanoid | Category: Technical Support Response

Platform Model: EngineAI T800 Dev Edition (AGX Orin + Nezha)	**Perception Sensors:** Odin2 RGBD / Huanmai 4-Mic Array / Livox LiDAR
Control Framework: Native SDK (MuJoCo / ROS2)	**Response Date:** July 2026

SECTION 1: VIDEO STREAMING & REAL-TIME INTEGRATION

QUESTION (0) - Live Video Streaming Procedure & Zero-Impact Integration

Customer Inquiry: What is EngineAI's officially supported procedure for streaming video on the T800? For an upcoming event, we aim to integrate the onboard cameras with our existing infrastructure to capture and transmit live video feed to our backend (e.g., for facial recognition). The target video resolution is 1080p (1920x1080). Crucially, this integration must not impact or degrade the robot's primary control systems, including balancing, locomotion, or performance routines.

Technical Answer & Integration Strategy:

- **Video Stream Access Procedure:** Video streaming is fully supported and managed via the `odin_ros_driver_node` node running on the NVIDIA AGX Orin high-performance compute unit (IP: `192.168.0.162`). You can SSH into the AGX Orin unit to launch the driver.
- **Published Topics:** Upon driver launch, the system automatically publishes live left and right camera streams over ROS/ROS2 topics:
  - `camera0/compressed`: `sensor_msgs/CompressedImage`, left camera JPEG compressed stream, recommended for network transmission to backend infrastructure.
  - `camera1/compressed`: `sensor_msgs/CompressedImage`, right camera JPEG compressed stream.
  - `camera0/undistort`: `sensor_msgs/Image` (RGB), uncompressed/rectified RGB image stream.
- **Resolution & Configuration:** Supported typical resolutions include 1280x1088 or 640x544 at 30 FPS (`mjpeg` format). You can launch the driver with custom parameters:

```
ros2 launch odin_ros_driver_rev1 driver.launch.py image_width:=640 image_height:=544
image_fps:=30 image_format:=mjpeg
```

- **Resolution Capability Note:** The requested 1080p (1920x1080) resolution is not listed in the default capability matrix table. Upon driver startup, the system calls `sdk::GetSensorCapability()` to query active hardware capabilities. Please verify supported resolutions at runtime.
- **Control System Isolation Guarantee:** T800 features a dual-computer isolated architecture: Motion Control Module (Intel Nezha N97, IP: `192.168.0.163`) and Application Computing Unit (NVIDIA AGX Orin, IP: `192.168.0.162`), bridged via an internal Gigabit router. All vision capture, encoding, and streaming algorithms run strictly on the AGX Orin. They consume zero CPU/GPU cycles on the motion controller, guaranteeing absolute zero degradation or impact on locomotion, balancing, or performance routines.

SECTION 2: NAVIGATION, KINEMATICS & MAINTENANCE

QUESTION (1) - Movement Interference & Obstacle Response

Customer Inquiry: Question regarding interference in the robot's movement: if something appears in front of the robot within a 3x3m space while we are moving-whether dancing or walking around the environment using the controller-what will happen? (Does it stop or continue?), or does it only obey the controller's command?

Technical Answer & Integration Strategy:

- **Behavior in Teleoperation Mode:** Under teleoperation / gamepad control modes (e.g., `walk` or `dance`), the underlying Finite State Machine (FSM) strictly executes commands received from the controller. Unless high-level SLAM mapping (via Odin2 / Livox Mid-360S) and autonomous obstacle avoidance nodes are explicitly activated on the AGX Orin, the robot will NOT automatically stop or steer around obstacles; it will continue executing controller movement commands.
- **Global Emergency Stop Mechanism:** At any moment during walking or dancing routines, operators can trigger a soft emergency stop by pressing the gamepad combination `LB + RB`. This instantly forces the robot into passive damping mode, disabling active motor torque to ensure human and hardware safety.

QUESTION (2) - Initial Startup Protocol & Joint Maintenance

Customer Inquiry: Regarding the initial startup following a period of storage or transport: is there a specific protocol to follow? Specifically, do the joints require lubrication or manual articulation while the robot is powered off before turning it on?

Technical Answer & Integration Strategy:

- **Standard Deployment Protocol:**
  1. Suspend the robot using a hoist/winch rigging harness.
  2. Power on the robot and press `LB + A` to enter `pd_stand` (PD standing control) mode.
  3. Once standing balance and ground contact are established, lower the robot smoothly onto flat ground.
  4. Press `LB + B` (walk) or `LB + CROSS_X_DOWN` (dance) to initiate desired locomotion routines.
- **Maintenance & Lubrication:** No manual joint articulation or lubrication is required while powered off. T800 joint modules employ proprietary Q-series servo actuators (Q300H, Q200H, Q50H, Q25H) with dual encoders and sealed planetary/harmonic reducers, which are factory-lubricated for lifetime operation.

QUESTION (3) - Low-Battery Locking State & Battery Swap Protocol

Customer Inquiry: When the robot signals a depleted battery, it locks in a standing position with rigid motors and a red LED on its head. In this case, will we need to transport the hoist/winch to the robot, or can it still be walked over to the hoist for battery removal?

Technical Answer & Integration Strategy:

- **Low-Battery Lock Protection:** When a depleted battery condition is triggered, the system automatically locks all joint motors in a rigid standing stance and turns on red head status LEDs. Teleoperation walking is disabled to prevent battery over-discharge and mechanical instability.
- **Field Handling & Battery Swap:** You MUST transport the hoist/winch to the robot's location. After securing the hoisting harness, use the quick-swap mechanism to detach the 1.22 kWh (7.2 kg) ternary lithium battery pack. Swap in a fresh battery pack or charge the depleted battery pack using the standalone charger (the robot body does not support direct plug-in charging).

> **QUESTION (4) - Hoist Specifications & Clear Height**
>
> *Customer Inquiry: What is the recommended clear height for the hoist, measured from the floor to the jib/boom? Additionally, regarding load capacity, is a 150 kg unit suitable, or is a higher-capacity hoist required?*

Technical Answer & Integration Strategy:

- **Hoist Load Capacity:** A 150 kg hoist unit is FULLY SUITABLE and offers a comfortable safety margin. The T800 Development Edition total mass with battery is ~84.5 kg (~7.2 kg battery), yielding a ~1.77x safety factor for hoisting, battery swapping, and fall protection.
- **Recommended Clear Height:** The standing height of T800 is 173 cm (1.73 m). Factoring in hoisting harness/rigging length (~30–50 cm) and vertical lift clearance (~30–50 cm), the recommended clear height from floor to jib/boom is at least 2.5 m to 2.8 m.

SECTION 3: AUDIO, HARDWARE APIS & LOW-LEVEL CONTROL

> **QUESTION (5) - Audio Streams & Native APIs**
>
> *Customer Inquiry: How do we access the raw audio streams and native APIs for the Huanmai 4-microphone array and speakers, considering they are absent from the topics published by the hardware controller?*

Technical Answer & Integration Strategy:

- **Hardware Overview:** T800 is equipped with a head-mounted Huanmai 4-microphone circular array (supporting voice wakeup, sound localization, and event detection) and torso-mounted dual hi-fi speakers.
- **System Bus Architecture:** The hardware motion controller ROS2 node (`hardware_interface_node`) focuses on IMU, joint states, and gamepad data. The audio devices are connected directly to the AGX Orin system bus and recognized as standard Linux ALSA / PulseAudio sound cards.
- **Native API Access:** Developers can directly access 4-channel raw PCM audio streams and speaker output via standard Linux C/C++ ALSA APIs (`alsa-lib`), PulseAudio, or Python audio libraries (`PyAudio`, `sounddevice`, `PortAudio`) on the AGX Orin.

> **QUESTION (6) - Zero-Copy DMA Access for RGBD Frames**
>
> *Customer Inquiry: Is there support for Zero-Copy direct memory access of the RGBD depth camera frames via the AGX Orin GPU, thereby completely bypassing ROS2 topic latency?*

Technical Answer & Integration Strategy:

- **High-Performance Depth Projection:** Odin2 ROS driver provides a header-only projection core (`pointcloud_to_depth_node.hpp`) to project raw dToF point clouds (`cloud/raw`) and rectified images (`camera0/undistort`) into 32FC1 dense depth maps directly within C++ applications.
- **GPU Zero-Copy DMA Support:** AGX Orin 64G utilizes Unified Memory Architecture (shared 64GB LPDDR5 RAM between CPU and GPU). The Odin C/C++ SDK (`sdk_api`) exposes raw memory buffer pointers. Developers can perform CUDA Zero-Copy pointer mapping (`cudaHostGetDevicePointer`) in C++/CUDA applications to pass camera and depth buffers directly to GPU kernels without ROS2 DDS serialization and IPC overhead, achieving sub-millisecond latency.

> **QUESTION (7) - Low-Level Control Loop & Neural Network Injection**
>
> *Customer Inquiry: How can we inject our neural network (PPO) directly into the lowest-level control loop, bypassing the standard state machine, without triggering a fatal system error?*

Technical Answer & Integration Strategy:

- **Core Architecture Principle:** Do NOT attempt to bypass the state machine directly. Bypassing it disables safety limits, kinematic sanity checks, and emergency fallbacks, which will trigger fatal system errors or hardware damage. Instead, integrate your PPO policy as a custom `MotionRunner` within the Native SDK state machine framework.
- **Step 1: Create Custom Runner:** Create a class inheriting from `MotionRunner` (e.g., `MyPPORunner`) and register it via `REGISTER_RUNNER(MyPPORunner, "ppo_runner", kMotion)`.
- **Step 2: Configure Control Context:** If your PPO model is trained on serial joint representations, set `data_store_->parallel_by_classic_parser.store(false)` inside `SetupContext()`.
- **Step 3: Initialize MNN Engine:** In the `Enter()` lifecycle stage, instantiate the model using the built-in `math::MNNModel` inference engine.
- **Step 4: Observation & Forward Inference:** In the 500 Hz control loop (period: 0.002s), extract joint positions/velocities (q, qd), IMU orientation/angular velocity, and filtered gamepad commands from `data_store_` to construct the observation vector and run forward inference.
- **Step 5: Safe Command Output:** Pass target joint actions to `GetMutableOutput().SetCommand(q_des, qd_des, Kp, Kd, tau_ff_des)`. The Task Manager performs smoothing, singularity checks, and torque limits before safely dispatching commands to motor drivers.

> **QUESTION (8) - Odin2 Point Cloud Extraction & ROS2 Stream Config**
>
> *Customer Inquiry: How do we extract the point cloud from the Odin2 depth camera, and what is the direct stream configuration from the Jetson Orin to ROS2?*

Technical Answer & Integration Strategy:

- **Point Cloud Extraction:** SSH into AGX Orin (192.168.0.162) and launch: `ros2 launch odin_ros_driver_rev1 driver.launch.py`. The driver publishes raw dToF point cloud `cloud/raw` (256x192, `sensor_msgs/PointCloud2`) and SLAM point cloud `cloud/slam`. Run `post_process_node` to enable colorized point clouds (`cloud/render`).
- **Stream Configuration File:** Edit configuration at `/home/ubuntu/odin/src/ros_driver/config/control_command.yaml` under `register_keys`:

```
register_keys:
  raw_point: true   # Enable raw dToF point cloud
  slam_point: true  # Enable SLAM point cloud
  image0: true      # Enable left camera stream
  image1: true      # Enable right camera stream
  imu: true         # Enable IMU stream
  odom: true        # Enable odometry stream
```

> **QUESTION (9) - Recommended Simulator & Math::MNNModel Deployment**
>
> *Customer Inquiry: Which simulator is recommended for policy training (e.g., Isaac Lab or MuJoCo), and what is the standard procedure for exporting the models for inference within Math::MNNModel?*

Technical Answer & Integration Strategy:

- **Recommended Simulator:** MuJoCo is natively supported and strongly recommended by the official Native SDK. The SDK provides build and execution scripts (`./scripts/build_mujoco.sh` and `./scripts/run_mujoco.sh`), complete URDF kinematics models matching real hardware, and Docker container support with NVIDIA GPU passthrough.

- **Step 1: Export MNN Model:** Export trained PyTorch policy to `.mnn` format (e.g., `policy.mnn`) and place it in `assets/resource/` or designated config directory.
- **Step 2: YAML Parameter Linking:** Declare `policy_file: "demo/policies/policy.mnn"` in the runner's YAML parameter file.
- **Step 3: Instantiate MNN Engine:** In `Runner::Enter()`, instantiate engine: `mlp_net_ = std::make_unique<math::MNNModel>(common::PathJoin(common::GlobalPathManager::GetInstance().GetC param_->policy_file));`
- **Step 4: Real-time Forward Inference:** In `Runner::Run()`, feed observation vector into `mlp_net_` for forward pass and output target actions to `SetCommand()`.